

AI SW·어플리케이션 품질검증 특강 · 2일차

AI 어플리케이션 품질검증 프레임워크 실습

테스트 자동화 도구와 AI 모델 검증 라이브러리로 하는 품질 보증

2026. 10. 2. (금) 09:00 ~ 18:00

1교시 시작 · 어제를 돌아보며

어제 헛갈렸던 것부터 짚고 시작합니다

어제 회고에서 나온 질문

- [회고 질문 1]
- [회고 질문 2]
- [회고 질문 3]

오늘의 계획

- 오전: 자동 테스트 · 테스트 케이스 · AI 결과 검증
- 오후: 교차 검증 · 리포트 · CI · 버그 찾기
- 8교시: 발표 · Kahoot · 회고

[] 칸은 1일차 회고 포스트잇을 보고 수업 전에 채웁니다.

오늘의 흐름

2일차 시간표

교시	시간	주제	활동
1	09:10	테스트 자동화: unittest·PyTest	자동 테스트 실행, 결과표 읽기
2	10:10	테스트 케이스 설계와 데이터 기반 테스트	표에 경계값 추가 → 결함 찾기
3	11:10	AI 결과 검증과 품질 게이트	손 계산 vs 도구, 기준 정하기
4	13:00	교차 검증	슬라이더로 실험, 배포 후보 고르기
5	14:00	성능 리포트 자동 생성	숫자 대조, 리포트 개선안
6	15:00	CI/CD 연동 테스트 (GitHub Actions)	강사 시연, Actions 화면 읽기
7	16:00	버그 리포팅과 이슈 트래커	버그 찾기 → 이슈 등록
8	17:00	종합 발표와 정리	팀 발표, Kahoot, 회고

오늘의 미션

자동 테스트 28개 모두 통과. 그런데 결함이 숨어 있습니다.

오늘은 코드를 고치지 않습니다. 무엇을 확인할지 정하고, 결과를 의심하며 읽고, 찾은 것을 정확히 보고합니다.
그것이 PM의 품질검증입니다.

오늘 쓰는 도구

코드 대신 입력 칸, 표, 화면을 다룹니다

도구	하는 일	교시
노트북 01 (Colab)	자동 테스트 실행, 테스트 케이스 표, 손 계산 비교, 품질 게이트, 리포트	1·2·3·5
노트북 02 (Colab)	교차 검증 실험 (슬라이더·목록)	4
GitHub Actions 화면	자동 테스트 실행 기록 읽기	6
ai-quality-bug-bash	가상 서비스 화면에서 버그 찾기, 이슈 등록	7

실습 안내: github.com/leejaehee1/ai-quality-lab → day2/README.md · 2인 1조 2팀(A팀, B팀)

사람 대신 코드가 확인합니다

	수동 테스트	자동화 테스트
실행	사람이 매번 실행하고 눈으로 확인	코드가 실행하고 결과를 판정
속도	느리고 빠뜨리기 쉬움	몇 초 만에 수백 개를 똑같이
변경할 때	다시 하기 어려움	바꿀 때마다 자동 실행 (회귀 테스트)

회귀 테스트: 고친 뒤에도 기존 기능이 여전히 되는지 확인하는 것. 자동화의 가장 큰 가치입니다.

반복되고, 기준이 명확하고, 사람이 실수하기 쉬운 것

자동화한다

- 전처리 함수의 입력·출력
- 데이터 검증 (결측·범위·중복)
- 모델 성능 하한, 강건성
- 리포트 생성

사람이 한다

- 탐색적 테스트
- 사용성 평가
- 결과 해석과 출시 판단

ML은 데이터가 바뀌거나 재학습할 때마다 다시 확인해야 해서 자동화가 더 중요합니다.

1교시 · 노트북 준비

▶ 버튼과 입력 칸만 씁니다

- 1 열기**
README의 노트북 01 열기 링크 → Colab
- 3 구성 보기**
칸마다 제목 · ▶ 버튼 · 오른쪽 입력 칸. 코드는 숨겨져 있습니다

- 2 준비**
0번 칸 ▶ → 경고 창은 '무시하고 실행' → '준비 완료'
- 4 역할 나누기**
한 명은 조작, 한 명은 기록. 교시마다 바꿉니다

런타임을 다시 시작하면 0번부터 다시 ▶ 합니다.

테스트 한 개 = 입력 + 기대 결과

```
# unittest (Python 기본 내장)
class TestIsEligible(unittest.TestCase):
    def test_typical_eligible(self):
        self.assertTrue(is_eligible(25, 12, False))    # 25세, 12개월, 미취업 → 대상

# PyTest (가장 많이 쓰는 도구)
def test_too_old():
    assert is_eligible(40, 12, False) is False        # 40세 → 대상 아님
```

test_로 시작하는 이름

테스트 하나

괄호 안의 값

입력 (나이, 거주 개월, 취업 여부)

assert

'이래야 한다'는 기대 결과

1교시 · 실습 10분

자동 테스트 전체 실행, 결과표 읽기

노트북 1번 칸 ▶

- '전체 28개 중 통과 28개'
- 영역: 무엇을 검사하나
- 확인하는 것 → 결과  / 

팀 토론 2분

- 모두  = 결함 없음일까?
- 표에 없는 것은 무엇인가?
- 1일차 테스트 원칙 1.7

테스트는 결함이 있다는 것을 보여 줄 뿐, 없다는 것을 증명하지 못합니다.

FIRST와 AAA

FIRST

- Fast: 빠르게
- Independent: 서로 독립적으로
- Repeatable: 언제 돌려도 같은 결과
- Self-validating: 스스로 판정
- Timely: 제때 작성

AAA 구조

- Arrange: 준비
- Act: 실행
- Assert: 확인

ML은 무작위성 때문에 Repeatable이 어렵습니다 → random_state 고정

PyTest는 unittest 테스트도 실행합니다

	unittest	PyTest
설치	표준 라이브러리	별도 설치
형태	클래스 + assert 메서드	함수 + 일반 assert 문
매개변수화	subTest (불편)	@pytest.mark.parametrize
준비 코드	setUp	fixture (재사용·범위 지정)

요구사항과 다른 판정을 찾으세요

요구사항

- 만 19세 이상 34세 이하
- 해당 지역 거주 6개월 이상
- 미취업

세 조건을 모두 만족하면 '대상'

노트북 2-2 칸

- 나이·거주 기간 슬라이더
- 취업 여부 목록
- ▶ → 서비스 판정

힌트: 1일차 경계값 18 · 19 · 34 · 35 / 5 · 6개월

이것이 탐색적 테스트입니다. 이상한 결과는 말하지 말고 메모만 해 두세요.

표의 한 행이 자동 테스트 하나입니다

case_id	age	months	employed	expected	reason
TC-01	25	12	아니오	대상	모든 조건 충족
TC-02	40	12	아니오	비대상	나이 초과
TC-03	25	3	아니오	비대상	거주 기간 부족
TC-04	25	12	예	비대상	취업 상태

```
@pytest.mark.parametrize("case", load_cases())    # 표의 행마다 반복
def test_case(case):
    assert is_eligible(나이, 거주, 취업) == 기대_결과
```

개발자는 표를 실행하는 장치를 한 번만 만들고, 케이스는 PM·기획자·QA가 표로 씁니다 (데이터 기반 테스트).

경계값 행을 설계하고 추가합니다

① 설계 (5분, 종이)

나이	거주	취업	기대 결과
18	12	아니오	?
19	12	아니오	?
34	12	아니오	?
35	12	아니오	?
25	5	아니오	?
25	6	아니오	?

② 추가와 실행

- 2-4 칸: 값 입력 → ▶ (행마다)
- 2-5 칸: 표만 실행 →  / 
- 잘못 넣으면 2-6 칸으로 지우기
-  행은 캡처해 두기 (7교시)

기대 결과는 서비스의 답이 아니라 요구사항으로 적습니다.

2교시 · 실행 결과

18세 행에서 ✕.
결함 1을 찾았습니다.

어제 슬라이드의 '커버리지 100%여도 남는 결함'과 같은 예입니다. 처음 4개 행으로는 왜 못 찾았을까요?

기대와 실제를 그대로 옮기면 버그 리포트가 됩니다

TC-05: 기대 '비대상', 실제 '대상' (나이 18, 거주 12개월, 취업 아시오)

기대 '비대상'

요구사항(만 19세 이상)에 따른 결과

실제 '대상'

서비스가 실제로 낸 결과

괄호 안

재현에 필요한 입력값 전부

고치는 것은 개발자, 고쳐졌는지 확인할 기준(이 행)을 남기는 것은 PM입니다.

얼마나 실행했나 ≠ 얼마나 맞나

파일	줄 수	실행 안 된 줄	커버리지
preprocess.py (대상 판정)	16	0	100%
evaluate.py (지표 계산)	15	0	100%
report.py (리포트)	40	2	95%
전체	99	2	98%

판정 코드는 커버리지 100%인데 결함 1이 있었습니다. 왜일까요?

모든 줄을 실행해도 경계값을 확인하지 않으면

커버리지가 알려 주는 것

어떤 줄이 한 번도 실행되지 않았는가 (Missing)

알려 주지 않는 것

실행한 줄이 올바르게 동작하는가. $18 \leq \text{age}$ 는 실행됐지만 틀렸습니다.

--cov-branch를 붙이면 분기 커버리지를 측정합니다.

지표 비교에는 approx

기능	예	용도
pytest.approx	assert 0.1 + 0.2 == pytest.approx(0.3)	실수 비교
tmp_path	generate_report(out_dir=tmp_path)	임시 폴더 파일 테스트
xfail	@pytest.mark.xfail(reason="이슈 #3")	알려진 결함 표시
마커	@pytest.mark.slow, -m "not slow"	느린 테스트 분리

0.1 + 0.2 == 0.3은 False입니다. 지표 값은 == 대신 approx로 비교합니다.

데이터 · 코드 · 모델

층	무엇을	1교시 결과표의 영역
데이터 테스트	결측, 범위, 범주, 중복, 분할 누수	데이터 품질 검사
코드 테스트	전처리·지표 계산이 명세대로	대상 판정 규칙, 결측값 대체, 지표 계산
모델 테스트	성능 하한, 기준선 대비, 불변성, 강건성	모델 품질 게이트

모델 테스트는 '정확히 이 값'이 아니라 '이 이상'을 확인합니다. 재학습하면 점수가 조금씩 바뀌기 때문입니다.

3교시 · 손 계산 ① 결측값 대체

빈칸은 학습 데이터의 중앙값으로 채웁니다

학습 데이터 · 나이
20, 30, 40, (빈칸)

테스트 데이터 · 나이
70, 80, (빈칸)

- 노트북 3-1 칸에 손으로 계산한 학습 중앙값을 입력 → ►
- 도구 결과와 다르면 **✗** → 결함 후보

테스트 데이터가 계산에 섞이면 '시험 문제를 미리 본' 셈입니다 (1일차 5교시 데이터 누수).

AI가 '악성'이라고 한 사람 중 실제 악성의 비율

환자	P1	P2	P3	P4	P5	P6	P7	P8
실제	악성	악성	악성	양성	양성	양성	양성	악성
AI 예측	악성	악성	악성	악성	악성	양성	양성	양성

- 찾으려는 대상(양성 클래스) = 악성
- 'AI 예측 = 악성'인 칸에 표시하고 그중 실제 악성을 세기
- 노트북 3-2 칸에 소수로 입력 (예: 0.75) → ►

도구가 틀렸나, 내가 틀렸나? 짝과 따로 계산해 비교합니다 (독립 검증).

이 기준을 못 넘으면 배포하지 않는다는 팀의 약속

0.956

현재 모델의 교차 검증 재현율 (± 0.025)

0.90

지금의 기준 (최소 재현율)

?

우리 팀의 기준

- 노트북 3-3 칸: 슬라이더로 기준을 올려 보며 **✗**(배포 불가)가 되는 지점 찾기
- 기준값은 PM이 근거를 갖고 정하고 책임지는 값입니다

3교시 · 팀 토론 8분

우리 서비스라면 기준을 얼마로?

암 진단 보조

악성을 놓치면 치명적. 최소 재현율은? 평균만 볼까, 평균 - 표준편차(0.931)도 볼까?

쇼핑 추천

하나 놓쳐도 큰 피해 없음. 같은 기준이 필요할까?

팀별 기준과 근거를 한 문장으로 화이트보드에 적습니다. 8교시 발표에 씁니다.

4교시 · 한 번의 분할은 운이다

같은 모델, 분할만 바꿔 20번 평가 (노트북 02)

0.888

최저 테스트 정확도

0.958

최고 테스트 정확도

0.929

평균

점수 하나로 '모델 A가 B보다 낫다'고 말하기 어렵습니다.

모든 데이터가 한 번씩 검증에 쓰입니다

폴드 1: [검증] [학습] [학습] [학습] [학습]

폴드 2: [학습] [검증] [학습] [학습] [학습]

폴드 3: [학습] [학습] [검증] [학습] [학습]

폴드 4: [학습] [학습] [학습] [검증] [학습]

폴드 5: [학습] [학습] [학습] [학습] [검증]

→ 5개 점수의 평균 $\pm$ 표준편차

교차 검증은 학습 데이터 안에서 합니다. 테스트 데이터는 여전히 마지막에 한 번만.

라벨 순으로 정렬된 데이터, 폴드별 악성 비율 (원본 약 0.37)

폴드	KFold (섞지 않음)	StratifiedKFold
1	1.00	0.38
2	0.86	0.38
3	0.00	0.37
4	0.00	0.37
5	0.00	0.37

분류 문제에는 StratifiedKFold를 씁니다.

슬라이더와 목록으로 실험합니다

칸	바뀌 볼 것	확인할 것
1. 분할 흔들림	측정 횟수, 트리 깊이	최저 · 최고 · 평균
2. 폴드별 악성 비율	(▶만)	KFold vs StratifiedKFold
3. 교차 검증	K (3·5·10), 모델 (3종)	평균 ± 표준편차, 폴드별 점수
4. 누수 실험 (선택)	(▶만)	누수 0.90 vs 올바른 방법 0.57

평균 ± 표준편차로 말합니다

지표	평균	표준편차
accuracy	0.979	0.009
precision	0.987	0.016
recall	0.956	0.025
f1	0.971	0.012

"재현율 0.956 ± 0.025 (5겹 층화 교차 검증, 학습 데이터 426건)"

배포 후보 모델을 고르세요

모델	K	재현율 평균 ± 표준편차	품질 게이트 통과?
로지스틱 회귀	5	0.956 ± 0.025	
결정 트리 (깊이 3)			
결정 트리 (제한 없음)	10		

3교시에 정한 우리 팀의 기준으로 판정합니다. 평균은 넘는데 흔들림이 크다면?

데이터의 구조에 맞게

방법	언제
GroupKFold	같은 환자의 데이터가 여러 개일 때 (중복 누수 방지)
TimeSeriesSplit	시간 순서가 있을 때 (과거로 학습, 미래로 검증)
K = 3 / 5·10 / 데이터 수	빠름·흔들림 큼 / 흔한 기본값 / 매우 느림

5교시 · 왜 자동으로 만드나

같은 형식으로 남겨야 비교할 수 있습니다

비교

재학습할 때마다 같은 형식의 결과

정확

손으로 복사하면 틀리고 빠뜨림

보관

CI가 매번 만들어 보관 (6교시)

5교시 · 리포트 한 번에 만들기

노트북 01 · 4번 칸 ▶

리포트 구성	읽는 법
교차 검증 표	지표별 평균 · 표준편차 · 폴드별 점수
테스트 데이터 평가 표	마지막에 한 번 평가한 점수
혼동 행렬 그림	실제 × 예측 건수. 지표를 직접 계산할 수 있음
classification_report	클래스(악성·양성)별 정밀도·재현율

과제: 혼동 행렬로 악성 정밀도를 직접 계산해 표의 precision과 비교하세요.

만드는 것보다 읽고 대조하는 것이 중요합니다

위치	악성 precision
표 2: 테스트 데이터 평가 (compute_metrics)	0.989
classification_report의 malignant 행	0.981

같은 모델, 같은 테스트 데이터인데 값이 다릅니다. 무엇을 의미할까요?

숫자가 서로 맞는지부터

- 같은 지표가 여러 곳에 있다면 값이 일치하는가
- 교차 검증 평균과 테스트 점수가 비슷한가 (크게 다르면 과적합·누수 의심)
- 튜는 폴드가 있는가
- 품질 기준(재현율 0.90 이상 등)을 만족하는가
- 데이터 건수, 양성 클래스 정의, 모델 설정이 적혀 있는가 (재현 가능성)

5교시 · 리포트 개선안 25분

PM·경영진이 읽는다면 무엇이 달라야 할까

1 읽는 사람과 결정

누가 이 리포트로 무엇을 결정하나

3 품질 게이트 판정

배포 가능 · 조건부 · 불가

2 첫 화면 숫자 3개

쉬운 말로: '악성 환자 53명 중 52명을 찾아냄'

4 이상한 점 · 빠진 정보

그룹별 성능, 이전 버전 비교, 데이터 기간

양식: `day2/templates/report_redesign.md` · 팀별 작성 15분 → 팀별 3분 공유

올릴 때마다 자동으로 확인

용어	뜻
CI (지속적 통합)	코드를 올릴 때마다 자동으로 빌드·테스트
CD (지속적 배포·전달)	통과한 코드를 자동으로 배포(또는 배포 준비)
품질 게이트	기준을 통과하지 못하면 병합·배포를 막는 관문

ML에서는 성능 기준을 게이트에 넣어, 성능이 떨어진 모델이 배포되지 않게 막습니다.

언제 · 어디서 · 무엇을

언제 (on)

- 코드를 올릴 때 (push)
- PR을 만들 때
- Run workflow 버튼

어디서 (runs-on)

- GitHub가 빌려주는 가상 컴퓨터

무엇을 (steps)

- 코드 내려받기 · Python 설치
- 테스트 + 품질 게이트 ← 실패하면 ❌
- 성능 리포트 (실패해도 항상)
- 실행 요약에 리포트 표시

파일: `.github/workflows/day2-ci.yml` · 문법은 몰라도 됩니다. 이 그림과 짝만 맞춰 봅시다.

다섯 개만 알면 읽을 수 있습니다

용어	뜻
workflow	자동화 전체 (YAML 파일 하나)
event (on)	실행 계기: push, pull_request, 수동 실행
job	가상 머신 하나에서 실행되는 작업 묶음
step	job 안의 한 단계: uses(만들어진 액션) 또는 run(명령)
artifact	실행 결과 파일 보관

if: always() → 테스트가 실패해도 리포트는 생성합니다. 실패했을 때 리포트가 더 필요하니까요.

표 한 줄과 기준값 하나가 배포를 멈춥니다

시연 A · 테스트 케이스 한 줄

- demo-ci 브랜치에서 표에 19세 행 추가 → 
- 18세 행 추가 → 
- 실패 단계 열기 → '기대 비대상, 실제 대상'

시연 B · 품질 게이트 기준

- Actions → Run workflow
- 재현율 기준 0.96 입력 → 
- 0.90 입력 → 

결함이 고쳐지기 전에는 아무도 배포할 수 없게 막는 것, 이것이 CI 품질 게이트입니다.

github.com/leejaehee1/ai-quality-lab/actions

찾을 것	위치
실행마다 성공·실패	목록의  / 
누가, 무엇을 바꿔서 실행됐나	실행 제목(커밋 메시지), 브랜치
어느 단계에서 실패했나	실행 → job → 빨간 단계 펼치기
성능 리포트	실행 → Summary 아래쪽
리포트 파일	Summary 맨 아래 Artifacts (내려받기는 로그인 필요)

질문: 빨간 불인 실행에도 리포트가 있나요? 왜 그럴까요?

병합하면 안 되는 상태

PR 화면

체크 결과가 표시되어 리뷰어가 바로 봅니다.

브랜치 보호 규칙

'상태 확인 통과 시에만 병합'을 켜면 빨간 불인 PR은 병합 버튼이 막힙니다. 이것이 품질 게이트입니다.

외부 텍스트를 명령에 넣지 않습니다

이슈 제목, PR 본문처럼 외부 사람이 쓴 텍스트를 `run:` 명령에 그대로 넣으면 명령어 주입 공격을 당할 수 있습니다. 필요하다면 `env`로 받아 따옴표로 감싸 씁니다. 오늘 워크플로는 그런 값을 쓰지 않습니다.

CI를 더 쓸모 있게

주제	내용
매트릭스 빌드	여러 Python 버전에서 동시에 테스트
캐시	pip 설치 시간 단축 (이미 적용)
배지	README에 CI 상태 표시
예약 실행	매일 밤 재학습·평가 (드리프트 감지)
CD	통과 시 모델 파일 자동 배포

누가 읽어도 똑같이 재현할 수 있게

구성	좋은 예	나쁜 예
요약	만 18세를 입력하면 지원 대상으로 판정함	판정 이상함
재현 절차	입력값을 번호 순서로	해 보면 틀려요
기대 결과	비대상 (R-01: 만 19세 이상)	없음
실제 결과	'지원 대상입니다' 표시, 캡처	이상하게 나옴
영향 (PM 관점)	대상 아닌 사람에게 지급 → 예산 손실	없음

사실과 추측을 구분합니다. 결함 하나에 이슈 하나.

영향과 긴급함은 다를 수 있습니다

	심각도 (Severity)	우선순위 (Priority)
질문	영향이 얼마나 큰가	얼마나 빨리 고쳐야 하나
주로 정하는 사람	테스터	관리자, 제품 책임자
예: 첫 화면 회사명 오타	낮음	높음
예: 거의 안 쓰는 기능 멈춤	높음	낮음

토론: 오늘 찾은 결함 3개의 심각도와 우선순위는?

7교시 · 버그 찾기 30분

github.com/leejaehee1/ai-quality-bug-bash

- 1 찾기 (12분)**
요구사항 명세(R-01~R-24)와 화면 4장을 한 줄씩 비교
- 3 증거**
캡처는 입력 칸에 끌어다 놓기. 오전 노트북의 **×**도 함께

- 2 등록 (18분)**
Issues → New issue → 버그 리포트. 제목 앞에 [A팀]
- 4 목표**
팀당 5개 이상. 개인 정보는 적지 않기

PM의 이슈가 개발자의 수정으로 이어집니다

트리아지 8분

- 다른 팀 이슈 읽기
- 중복이면 댓글 '#번호와 중복'
- 심각도·우선순위 의견 남기기

이슈 이후 (설명)

- 담당자 지정 → 브랜치에서 수정
- PR 본문에 Fixes #번호
- CI  → 병합 → 이슈 자동 종료

화면 ①의 18세 판정과 노트북의 TC-05 는 같은 결함입니다.

GitHub 외의 도구도 구성은 비슷합니다

기능	용도
라벨	bug, enhancement, 심각도별 분류·검색
담당자	누가 처리하는지
마일스톤	배포 버전 단위로 묶기
프로젝트 보드	할 일 → 진행 중 → 완료
이슈 템플릿	필수 정보가 빠지지 않게 (오늘 사용)

Jira, Redmine 등도 같은 개념을 씁니다.

8교시 · 팀 발표

팀당 8분

- 1 가장 심각한 결함**
무엇이고, 왜 가장 심각한가
- 3 리포트 개선안**
경영진 보고용 한 줄 요약

- 2 품질 게이트**
우리 팀의 기준값과 근거
- 4 배운 점**
종합 프로젝트에 적용할 한 가지

8교시 · Kahoot 퀴즈 12분

이틀 동안 배운 것, 12문제로 확인

1 접속
휴대폰으로 kahoot.it

2 PIN 입력
화면에 보이는 숫자

3 닉네임
팀 이름 + 이름

1일차 5문제 + 2일차 7문제 · 연결이 안 되면 다음 슬라이드의 퀴즈로 진행합니다.

문제 1~2

Q1. [1일차] 암 선별 검사에서 가장 중시할 지표는?

- A. 정확도
- B. 정밀도
- C. 재현율
- D. R^2

Q2. [1일차] 그룹별로 성능을 나눠 보는 이유는?

- A. 실행 속도를 높이려고
- B. 전체 성능 뒤에 숨은 소수 그룹 문제를 찾으려고
- C. 커버리지를 높이려고
- D. 비용을 줄이려고

문제 3~4

Q3. [1일차] 입력에 작은 잡음을 넣어 성능 하락을 보는 테스트는?

- A. 섭동 테스트
- B. 회귀 테스트
- C. 인수 테스트
- D. 통합 테스트

Q4. [1일차] R^2 가 0이라는 것은?

- A. 완벽한 예측
- B. 평균으로만 찍는 것과 같은 수준
- C. 가장 나쁜 예측
- D. 과적합

문제 5~6

Q5. [1일차] 테스트 데이터는 언제 쓰나?

- A. 학습 중 계속
- B. 설정값을 고를 때마다
- C. 마지막에 딱 한 번
- D. 전처리를 fit할 때

Q6. 테스트 케이스 표의 행마다 같은 테스트를 반복 실행하는 PyTest 기능은?

- A. fixture
- B. parametrize
- C. approx
- D. coverage

문제 7~8

Q7. 테스트 케이스 표의 '기대 결과'는 무엇을 기준으로 적나?

- A. 서비스가 실제로 낸 결과
- B. 요구사항·명세
- C. 개발자의 의견
- D. 이전 버전의 결과

Q8. 품질 게이트란?

- A. 기준을 통과하지 못하면 배포를 막는 관문
- B. 테스트 코드를 자동으로 고치는 도구
- C. 모델 학습 속도를 높이는 설정
- D. 이슈를 자동으로 닫는 기능

문제 9~10

Q9. 분류 문제에서 폴드마다 클래스 비율을 유지하는 방법은?

- A. KFold
- B. StratifiedKFold
- C. TimeSeriesSplit
- D. GroupKFold

Q10. GitHub Actions에서 테스트가 실패하면?

- A. 코드가 자동으로 고쳐진다
- B. 빨간 X로 표시된다
- C. 무시하고 넘어간다
- D. 자동으로 병합된다

문제 11~12

Q11. PR 본문에 'Fixes #3'을 쓰면?

- A. 3번 커밋이 삭제된다
- B. 병합할 때 3번 이슈가 자동으로 닫힌다
- C. 3번 테스트를 건너뛴다
- D. 3명이 리뷰해야 한다

Q12. 첫 화면의 회사 이름 오타, 심각도와 우선순위는?

- A. 높음 · 높음
- B. 낮음 · 높음
- C. 높음 · 낮음
- D. 낮음 · 낮음

기준을 이해하고, 도구로 확인했습니다

1일차 · 기준

- 테스트 원칙, 블랙박스·화이트박스
- 분류·회귀 지표와 해석
- 과적합·누수·편향·강건성
- 데이터 품질 검수 기준

2일차 · 확인

- 자동 테스트 실행과 결과 읽기
- 테스트 케이스 표·손 계산으로 결함 3개
- 교차 검증, 리포트 대조와 개선안
- CI 품질 게이트, 이슈로 결함 보고

8교시 · 이들의 회고 (KPT) 13분

포스트잇에 쓰고, 돌아가며 한 줄씩

Keep

좋았던 것, 계속하고 싶은 것

Problem

어려웠던 것, 아쉬웠던 것

Try

앞으로 해 볼 것

마지막 한 줄: 종합 프로젝트에 적용할 한 가지는?

마지막 메시지

테스트가 통과했다는 것은 결함이 없다는 뜻이 아닙니다

무엇을 기준으로 테스트했는지가 품질을 결정하고, 그 기준을 정하는 사람이 PM입니다.